Programming Assignment 3 (PA3)

Magical Demon Hunters Game
User-Defined Types, Encapsulation, Operator Overloading, Interacting Objects
Out: March 11, 2026 -- DUE: April 7, 2026
EC327 Introduction to Software Engineering – Fall 2026

Total: 200 points

- *You may use any development environment you wish, as long as it is ANSI C++ 11 compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many checkpoints as possible before the deadline. If you have missing implementations in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

Submission Format (Must Read)

- Use the **exact** file names specified in each problem for your solutions.
- Push your changes to the repository generated by GitHub Classroom.
- Complete submissions should have **22 files.**
- Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.
- **Code must compile in order to be graded. Otherwise, this is an automatic zero.**
- Comment your code (good practice!). We **may** use your comments when grading.

Coding Style (reminder from PA2) As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you begin to adopt your own. Naming conventions will help you recognize variable names, functions, constants, classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Good reference: https://google.github.io/styleguide/cppguide.html If you find any other good references, feel free to post the link on Piazza!

Class Specifications

Each class and its members are described below by listing its name, the prototypes for the member functions, and the names and types of the member variables. You **MUST** use the prototypes, types, and names in your program as specified here, in the same upper/lower case. Function argument names are allowed to be different. **Failure to follow these specifications will result in lost points.**

Checkpoint 1

Point2D (10 points)

This class contains two double values, which will be used to represent a set of (x, y) Cartesian coordinates. This class and `Vector2D` (described below) will be used to simplify keeping track of the coordinates of each object in the game, and updating their locations as they move. All data members and functions for this class should be **public**.

Public Members

- `double x`
  - The x value of the point.
- `double y`
  - The y value of the point.

Constructors

- The default constructor initializes x and y to 0.0
- `Point2D(double in_x, double in_y)`
  - Sets x and y to `in_x` and `in_y`, respectively.

Non-member Functions

- `double GetDistanceBetween(Point2D p1, Point2D p2)`
  - Returns the Cartesian (ordinary) distance between p1 and p2.

Non-member Overloaded Operators

Assume p1 and p2 represent two Point2D objects, and v1 represents a Vector2D object.

- Stream output operator (<<): produces output formatted as (x, y)
  - Example: If p1 has x = 3.14, y = 7.07 then cout << p1 will print (3.14, 7.07)
- Addition operator (+): p1 + v1 returns a `Point2D` object with x = p1.x + v1.x and y = p1.y + v1.y
  - Example: If p1 has x = 3, y = 7 and v1 has x=5, y=-2 then this function should make a new `Point2D` with x = 8 and y = 5;
- Subtraction operator (-): p1 - p2 returns a `Vector2D` object with x = p1.x - p2.x and y = p1.y - p2.y
  - Example: If p1 has x = 3, y = 7 and p2 has x=5, y=-2 then this function should make a new `Vector2D` with x = -2 and y = 9.

Vector2D

(10 points)

This class also contains two double values, but it is used to represent a vector in the real plane (a set of x and y displacements). The overloaded operators allow one to do simple linear-algebra operations to compute where an object's new location should be as it moves around. Some of the overloaded operators and other functions can be member functions, others cannot. All data members and functions for this class should be public.

Public Members

- `double x`
  - The x displacement value of the vector.
- `double y`
  - The y displacement value of the vector.

Constructors

- The default constructor that initializes x and y to 0.0
- `Vector2D(double in_x, double in_y)`
  - sets x and y to `in_x` and `in_y`, respectively.

Non-member Overloaded Operators (assume v1 represents a `Vector2D` object and d represents a non-zero double value)

- Multiplication operator (*): v1 * d returns a `Vector2D` object with x = v1.x *d and y = v1.y * d
  - Example: If v1 has x=10 and y=20 and d=5 then this function should make a new `Vector2D` with x=50 and y=100.
- Division operator (/): v1 / d returns a `Vector2D` object with x = v1.x / d and y = v1.y / d
  - Example - If v1 has x=10 and y=20 and d=5 then this function should make a new `Vector2D` with x=2 and y=4.
  - Dividing by zero should just create v1.
- Stream output operator (<<): produce output formatted as <x, y>
  - Example: If v1 has x = 5.3, y = 2.4 then `cout << v1` will print <5.3, 2.4>
  - Notice that this is < and > NOT (and) (like for `Point2D`).

GameObject

(20 points)

This class is the base class for all the objects in the game. It is responsible for the member variables and functions that they **all** have in common. It has the following members:

Protected members:

- `Point2D location;`
  - The location of the object
- `int id_num`
  - This object's ID
- `char display_code;`
  - How the object is represented in the View.
- `char state;`
  - State of the object; more information provided in each derived class.

Public members:

- `GameObject(char in_code);`
  - Initializes the `display_code` to `in_code`, `id_num` to 1, and `state` to 0. It outputs the message: **"GameObject constructed"**.
- `GameObject(Point2D in_loc, int in_id, char in_code,);`
  - Initializes the `display_code`, `id_num`, and `location`. The `state` should be 0. It outputs the message: **"GameObject constructed"**.
- `Point2D GetLocation();`
  - Returns the `location` for this object.
- `int GetId();`
  - Returns the `id` for this object
- `char GetState();`
  - Returns the state for this object.
- `void ShowStatus();`

  - Outputs the information contained in this class: `display_code`, `id_num`, `location`. i.e. **"(display_code)(id_num) at (location)"**. See sample output for exact formatting.

- (Notice that later you will be adding a few other functions including **pure virtual functions** called `Update()` and `ShouldBeVisible()` This will come when you learn about the `Model`).

CHECKPOINT I

Start by writing the `Point2D` and `Vector2D` classes and a couple of their functions. Add the additional functions one at a time and test each one. Write a `TestCheckpoint1.cpp` file with a main function to test them. There, create multiple `Point2D` and `Vector2D` objects in order to test their constructors and overloaded operators (<<, +, -, *, /). Getting the overloaded output operator to work soon will make testing the remaining functions more fun. Create `GameObject` and instantiate a few cases in your main function and run the `ShowStatus()` function. Convince yourself that the objects have been created correctly and that their member variables have the right values.

DO NOT MOVE ON UNTIL YOU ARE 100% SURE YOU UNDERSTAND THIS CHECKPOINT

If you do not make sure that these objects work fully, the rest of your game will NOT work. Use this time to also start to modify the provided makefile to also work for testing